

Actividad 1

Preprocesamiento y vocabulario de la colección Time

Bryan Ezequiel Violante Arriaga - Otoño 2026

Resumen

Se preprocesó la colección Time y se extrajo el vocabulario con su frecuencia de término, tanto de los 423 documentos como de las 83 consultas, utilizando el texto en crudo, sin palabras nulas, truncado con Porter original y con la variante de NLTK. El preprocesamiento consistió en eliminar las palabras vacías de la lista propia de la colección y truncar con el algoritmo de Porter. El vocabulario global pasó de 29 418 términos en el texto crudo a 16 675 tras el pipeline completo, una reducción del 43.3 %. También hicieron comparaciones de los vocabularios en las diferentes etapas del preprocesamiento.

1. La colección

Time es una colección de prueba clásica de recuperación de información, distribuida por el IR Group de Glasgow. Son artículos de la revista Time de 1963, en texto ASCII, todo en mayúsculas y con la puntuación mayormente separada por espacios.

Archivo	Contenido	Cantidad
TIME.ALL	Documentos, marcador *TEXT	423
TIME.QUE	Consultas, marcador *FIND	83
TIME.REL	Juicios de relevancia	83
TIME.STP	Palabras vacías	340

2. Método

El procesamiento se implementó en `actividad.ipynb`, un *notebook* de Jupyter sobre Python 3.13. Se apoya en dos bibliotecas y en nada más:

Biblioteca	Versión	Para qué
<code>nltk</code>	3.10.3	El <code>PorterStemmer</code> , en sus dos modos
<code>pandas</code>	3.0.5	Las tablas de vocabulario y la matriz documento-término

La lectura, la tokenización y el filtrado de palabras vacías se resolvieron con la biblioteca estándar, principalmente `collections.Counter` para las frecuencias y `str.strip` para la puntuación. Las versiones están fijadas en `../requirements.txt`, porque el resultado depende de ellas: el `PorterStemmer` de NLTK cambia de comportamiento entre modos, y eso mueve el vocabulario.

El procesamiento son cuatro etapas, aplicadas por igual a documentos y consultas:

1. **Lectura.** Se separa cada archivo por su marcador. Un documento va desde su `*TEXT` hasta el siguiente, o hasta `*STOP` al final.
2. **Tokenización.** Se parte por espacios y se quita la puntuación de los extremos de cada token, conservando la interna para no destrozarse siglas como *U.S.*

3. **Eliminación de palabras vacías.** Se comparan los tokens limpios contra las 340 entradas de `TIME.STP`.
4. **Truncamiento.** Porter, en sus dos variantes: el algoritmo original de 1980 y el de NLTK por omisión.

El vocabulario se representa en forma larga, una fila por par documento-término con su frecuencia, y se escribe al formato pedido: una línea por documento con su identificador seguido de los pares término y frecuencia.

3. Resultados

Después de aplicar cada etapa de procesamiento, el vocabulario de la colección se reduce progresivamente, como se muestra en la Tabla 1.

Etapas	Tokens	Vocabulario	Pares	Reducción
Texto crudo	263 749	29 418	140 756	—
Sin palabras vacías	124 030	23 338	96 865	20.7 %
Porter (1980)	124 030	16 675	92 091	28.6 %
Porter (NLTK)	124 030	16 623	92 040	28.8 %
Acumulado	−53.0 %	−43.3 %	−34.6 %	

Tabla 1: Efecto de cada etapa sobre los 423 documentos. La reducción de cada renglón es sobre el vocabulario del anterior; las dos variantes de Porter parten de la misma etapa. El acumulado es con Porter de 1980.

Y, al construir la matriz del lenguaje como lo vimos en clase, con los documentos como instancias y las palabras como atributos, se obtuvo una matriz dispersa con las frecuencias de cada término en cada documento, de dimensiones distintas según la etapa del preprocesamiento.

Etapas	Dimensiones	Celdas	No cero	Dispersión
Texto crudo	423 × 29 418	12 443 814	140 756	98.87 %
Sin palabras vacías	423 × 23 338	9 871 974	96 865	99.02 %
Porter (1980)	423 × 16 675	7 053 525	92 091	98.69 %
Porter (NLTK)	423 × 16 623	7 031 529	92 040	98.69 %

Tabla 2: Matriz documento-término en cada etapa. Las celdas distintas de cero son los pares documento-término que sí aparecen.

Más del 98 % de esa matriz son ceros en todas las etapas. En denso, con enteros de 32 bits, la del texto crudo pesa 49.8 MB y la de Porter 28.2 MB, casi puros ceros. Por eso el vocabulario se guarda en forma larga, que lista solo lo que sí aparece, y por eso el formato que pide el enunciado es esa misma forma.

A continuación se muestran los 15 términos más comunes en cada etapa (crudo, sin vacías, Porter de 1980 y Porter de NLTK), así como las discrepancias entre los dos modos de Porter.

Crudo		Sin vacías		Porter 1980		Porter NLTK	
<i>the</i>	15 809	<i>u.s</i>	726	<i>u.</i>	726	<i>u.</i>	726
<i>.</i>	12 148	<i>government</i>	583	<i>govern</i>	609	<i>govern</i>	609
<i>of</i>	7 219	<i>new</i>	549	<i>new</i>	603	<i>new</i>	549
<i>to</i>	6 303	<i>party</i>	378	<i>parti</i>	434	<i>parti</i>	434
<i>a</i>	5 781	<i>de</i>	335	<i>nation</i>	426	<i>nation</i>	426
<i>and</i>	5 501	<i>minister</i>	310	<i>communist</i>	420	<i>communist</i>	420
<i>in</i>	5 244	<i>west</i>	310	<i>red</i>	377	<i>red</i>	377
<i>"</i>	4 946	<i>war</i>	280	<i>minist</i>	370	<i>minist</i>	370
<i>that</i>	2 450	<i>south</i>	280	<i>de</i>	355	<i>de</i>	355
<i>for</i>	2 217	<i>communist</i>	278	<i>forc</i>	331	<i>forc</i>	331
<i>was</i>	2 134	<i>red</i>	277	<i>west</i>	310	<i>west</i>	310
<i>with</i>	1 830	<i>viet</i>	267	<i>war</i>	297	<i>war</i>	297
<i>his</i>	1 810	<i>british</i>	247	<i>polit</i>	281	<i>polit</i>	281
<i>is</i>	1 777	<i>president</i>	243	<i>south</i>	280	<i>south</i>	280
<i>he</i>	1 659	<i>world</i>	240	<i>countri</i>	270	<i>countri</i>	270

Tabla 3: Los 15 términos más frecuentes en cada etapa, con su número de apariciones en toda la colección.

Como se puede observar, en crudo los primeros lugares son artículos, preposiciones y el punto, que con 12 148 apariciones es el segundo término más frecuente de la colección, ninguno dice nada del contenido. Pero al quitar las vacías se puede vislumbrar el tema real, o al menos tratar de inferirlo. Y con Porter las frecuencias *suben* aunque el vocabulario baje, porque *govern* absorbe a *government* y *governments*.

Palabra	Porter 1980	NLTK	Veces
<i>days</i>	<i>dai</i>	<i>day</i>	136
<i>away</i>	<i>awai</i>	<i>away</i>	100
<i>money</i>	<i>monei</i>	<i>money</i>	61
<i>trying</i>	<i>try</i>	<i>tri</i>	60
<i>news</i>	<i>new</i>	<i>news</i>	54
<i>assembly</i>	<i>assembli</i>	<i>assembl</i>	45
<i>pay</i>	<i>pai</i>	<i>pay</i>	45
<i>stay</i>	<i>stai</i>	<i>stay</i>	42
<i>died</i>	<i>di</i>	<i>die</i>	40
<i>try</i>	<i>try</i>	<i>tri</i>	37
<i>eyes</i>	<i>ey</i>	<i>eye</i>	34
<i>possibly</i>	<i>possibli</i>	<i>possibl</i>	31
<i>key</i>	<i>kei</i>	<i>key</i>	31
<i>spy</i>	<i>spy</i>	<i>spi</i>	26
<i>age</i>	<i>ag</i>	<i>age</i>	24

Tabla 4: Las 15 discrepancias más frecuentes entre los dos modos de Porter, sobre los 124 030 tokens procesados.

De la misma manera se procesaron las queries, obteniendo diferentes representaciones dependiendo de la etapa de procesamiento.

Etapa	Tokens	Vocabulario	Pares
Texto crudo	1 341	552	1 228
Sin palabras vacías	722	449	697
Porter (1980)	722	413	695
Porter (NLTK)	722	413	695

Tabla 5: Lo mismo sobre las 83 consultas. Son mucho más cortas: entre 3 y 18 palabras tras quitar las vacías, media 8.7.

La eliminación de palabras vacías es la que se lleva más de la mitad de los *tokens* (53 %), pero apenas un quinto del vocabulario: quita muchas ocurrencias de pocas palabras distintas. Porter hace lo contrario, no cambia el número de tokens y encoge el vocabulario otro 28.6 %, porque solo agrupa formas.

4. Conclusiones

El pipeline reduce el vocabulario a menos de la mitad, de 29 418 a 16 675 términos, y las dos etapas contribuyen de forma distinta, quitar palabras vacías recorta ocurrencias, Porter agrupa formas.

Quedan dos artefactos sin resolver. Porter se come la *S* de *U.S* tomándola por plural y la convierte en *u.*, que además es el término más frecuente del documento 017, con 15 apariciones. Y el apóstrofo interno sobrevive al truncado de los extremos, así que *britain* y *britain'* quedan como dos términos distintos en el mismo documento.

Entregables

En `out/`, una línea por documento o consulta con el formato `Doc<id> término frecuencia ...`:

<code>vocab_raw.txt</code>	Texto crudo
<code>vocab_nn.txt</code>	Sin palabras vacías
<code>vocab_porter.txt</code>	Sin vacías y con Porter (1980)
<code>vocab_queries_*.txt</code>	Lo mismo para las consultas

El código está en `actividad.ipynb` y el entorno en `../requirements.txt`.

Referencias

- [1] **Colección Time.** IR Group, University of Glasgow. Colecciones de prueba para recuperación de información.
http://ir.dcs.gla.ac.uk/resources/test_collections/
De ahí se descargó `time.tar.gz`, con `TIME.ALL`, `TIME.QUE`, `TIME.REL` y `TIME.STP`.
- [2] **Algoritmo de Porter.** Martin Porter. Página oficial del algoritmo, con la especificación y las implementaciones.
<https://tartarus.org/martin/PorterStemmer/>
Publicado originalmente como *An algorithm for suffix stripping*, Program, 14(3), pp. 130–137, 1980.
- [3] **NLTK.** La implementación usada es `nltk.stem.PorterStemmer` 3.10.3, en sus modos `ORIGINAL_ALGORITHM` y `NLTK_EXTENSIONS`.